Getting Started with Gema Backend API

Welcome to the Gema Event Management Platform API! This guide will help you get up and running quickly.

Table of Contents

Prerequisites

Before you begin, ensure you have:

- **Node.js** v18 or higher
- **MongoDB** (local installation or Atlas cloud)
- **Firebase project** (for authentication)
- **Git** for version control

Quick Setup

1. Clone and Install

```
git clone <repository-url>
cd gema/backend
npm install
```

2. Environment Configuration

Create `.env` file:

```
# Copy the example file
cp .env.example .env
```

Essential Environment Variables:

```
# Server Configuration
PORT=5000
NODE_ENV=development

# Database
MONGODB_URI=mongodb://localhost:27017/gema

# JWT Configuration
JWT_SECRET=your_super_secure_jwt_secret_minimum_32_characters
JWT_EXPIRES_IN=7d
JWT_REFRESH_SECRET=your_refresh_secret_minimum_32_characters
JWT_REFRESH_EXPIRES_IN=30d

# Firebase Authentication
FIREBASE_PROJECT_ID=your-firebase-project-id
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxx@your-project.iam.gserviceaccount.com

# Frontend URL (for CORS)
FRONTEND_URL=http://localhost:5173
```

3. Database Setup

```
# Start MongoDB (if running locally)
mongod

# Seed initial data
npm run db:seed
```

4. Start Development Server

```
npm run dev
```

The API will be available at `http://localhost:5000`

First API Call

Test your setup with a simple health check:

```
curl http://localhost:5000/api
```

Expected Response:


```
{
  "message": "Welcome to the Gema API!"
}
```


Get Available Event Categories

```
curl http://localhost:5000/api/events/categories
```

Expected Response:


```
{
  "success": true,
  "message": "Event categories retrieved successfully",
  "data": {
    "categories": [
      "Art & Culture",
      "Business",
      "Family & Kids",
      "Music",
      "Sports & Recreation",
      "Technology"
    ]
  }
}
```


Authentication Flow

1. User Registration

```
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@example.com",
    "password": "SecurePassword123!",
    "phone": "+1234567890"
  }'
```

Response:

```json
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user": {
      "id": "user_id_here",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john.doe@example.com",
      "role": "customer"
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}
```

2. User Login

```bash
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "john.doe@example.com",
    "password": "SecurePassword123!"
  }'
```

3. Using Authentication Token

Include the token in subsequent requests:

```bash
curl -X GET http://localhost:5000/api/auth/me \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN_HERE"
```

Common Workflows

Creating an Event (Vendor)

1. **Register as vendor** or **login as existing vendor**
2. **Create event:**

```bash
curl -X POST http://localhost:5000/api/events \
  -H "Authorization: Bearer YOUR_VENDOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Amazing Music Concert",
    "description": "A wonderful evening of live music",
    "category": "Music",
    "type": "Event",
    "venueType": "Indoor",
    "ageRange": [18, 65],
    "location": {
      "city": "Dubai",
      "address": "123 Music Hall Street",
      "coordinates": {
        "lat": 25.2048,
        "lng": 55.2708
      }
    },
    "price": 150,
    "currency": "AED",
    "tags": ["music", "concert", "live"],
    "dateSchedule": [{
      "date": "2024-06-15T19:00:00Z",
      "availableSeats": 500,
      "price": 150
    }]
  }'
```

Placing an Order (Customer)

1. **Browse events** (no auth required):

```
curl "http://localhost:5000/api/events?category=Music&limit=5"
```

2. **Create order** (requires customer auth):

```
curl -X POST http://localhost:5000/api/orders \
  -H "Authorization: Bearer YOUR_CUSTOMER_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "items": [{
      "eventId": "EVENT_ID_FROM_BROWSE",
      "scheduleDate": "2024-06-15T19:00:00Z",
      "quantity": 2
    }],
    "billingAddress": {
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com",
      "phone": "+1234567890",
      "address": "123 Main St",
      "city": "Dubai",
      "state": "Dubai",
      "zipCode": "12345",
      "country": "UAE"
    }
  }'
```

File Upload Example

```
curl -X POST http://localhost:5000/api/uploads/single \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -F "file=@/path/to/your/image.jpg"
```

Testing with Different Roles

The seeded database includes test users:

```
# Admin User
POST /api/auth/login
{
  "email": "admin@gema.com",
  "password": "admin123"
}

# Vendor User
POST /api/auth/login
{
  "email": "vendor@gema.com",
  "password": "vendor123"
}

# Customer User
POST /api/auth/login
{
  "email": "customer@gema.com",
  "password": "customer123"
}
```

Error Handling

All API responses follow a consistent format:

Success Response:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... }
}
```

Error Response:

```
{
  "success": false,
  "message": "Error description",
  "error": {
    "code": "ERROR_CODE",
    "details": "Detailed error information"
  }
}
```

Common HTTP Status Codes:

- `200` - Success
- `201` - Created
- `400` - Bad Request (validation errors)
- `401` - Unauthorized (authentication required)
- `403` - Forbidden (insufficient permissions)
- `404` - Not Found
- `500` - Internal Server Error

Next Steps

Now that you're up and running:

1. **Explore API Reference**: Check out detailed endpoint documentation in [`../api/api-reference.md` (../api/api-reference.md)](../api/api-reference.md)
2. **Authentication Deep Dive**: Learn more about auth flows in [`../api/authentication.md` (../api/authentication.md)](../api/authentication.md)
3. **Deployment Guide**: Ready for production? See [`../deployment/deployment-guide.md` (../deployment/deployment-guide.md)](../deployment/deployment-guide.md)

Need Help?

- **Troubleshooting**: [`../troubleshooting/troubleshooting.md` (../troubleshooting/troubleshooting.md)](../troubleshooting/troubleshooting.md)
- **API Reference**: [`../api/api-reference.md` (../api/api-reference.md)](../api/api-reference.md)
- **Issues**: Report bugs or request features on GitHub

Quick Reference

Base URL: `http://localhost:5000/api` (development)

Key Endpoints:

- `GET /` - Health check
- `POST /auth/register` - User registration
- `POST /auth/login` - User login
- `GET /events` - Browse events (public)
- `POST /events` - Create event (vendor)
- `POST /orders` - Create order (customer)
- `GET /auth/me` - Get current user (authenticated)

Headers for Authenticated Requests:

```
Authorization: Bearer YOUR_ACCESS_TOKEN
Content-Type: application/json
```